

P0701r0

Back to the `std2::future`

Published Proposal, 18 June 2017

This version:

<http://wg21.link/P0701r0>

Author:

[Bryce Adelstein Leibach](#) (Lawrence Berkeley National Laboratory)

Audience:

SG1

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

1	Overview
2	Problems with the Existing Design
2.1	Single Consumer (<code>future</code>) vs Multiple Consumers (<code>shared_future</code>)
2.2	Concurrent Producer/Consumer(s) vs Non-Concurrent Producer/Consumer(s)
2.3	Coupling with Scheduling and Genericity
2.3.1	<code>future/promise</code> are Too Tightly Coupled to Scheduling
2.3.2	<code>future/promise</code> are Not Tightly Enough Coupled to Scheduling
2.3.3	Where Does a <code>.then</code> Continuation Run?
2.4	Composability
2.4.1	Passing <code>futures</code> to <code>.then</code> Continuations is Unwieldy
2.4.2	<code>when_all</code> and <code>when_any</code> Return Types are Unwieldy
2.4.3	Immediate Values
3	HPX/Berkeley Inspired Future

4 Acknowledgements

References

Informative References

§ 1. Overview

C++11 introduced asynchronous programming facilities:

- `future/shared_future` - Provides access to an asynchronous value.
- `promise` - Produces an asynchronous value.
- `async` - Runs a function asynchronously, producing an asynchronous value.
- `packaged_task` - Packages a function to store its result as an asynchronous value.

The Concurrency TS v1 extended these interfaces, adding:

- `future::then` - Attach continuations to asynchronous values.
- `when_all/when_any` - Combine and select asynchronous values.

I think many of us would agree with the following statements regarding these features:

- The programming model is correct.
- The current design is fatally flawed.

E.g. We want composable futures, but the ones we have today are broken. Fixing what we have today would be challenging without introducing backwards incompatible changes. So, the premise of this paper is that we should introduce new asynchronous programming primitives in conjunction with executors [\[P0443r1\]](#) instead of trying to "fix" what we have today.

This paper has two sections:

- First, I enumerate what I perceive to be problems with the current design.
- Then, I briefly describe a future design I've been toying around with, based some existing libraries I work on.

§ 2. Problems with the Existing Design

§ 2.1. Single Consumer (`future`) vs Multiple Consumers (`shared_future`)

When retrieving a value from a `future` (`.get`), we have two options:

- Move the value out of the shared state and return it, preventing any additional retrievals and avoiding copies. `future` has these semantics (33.6.7 [`futures.unique_futures`] p14-18).
- Return a reference, constant reference or copy of the value, allowing additional retrievals at the cost of copies. `shared_future` has these semantics (returns a reference or constant reference; 33.6.5 [`futures.shared_futures`] p16-20).

The difference in semantics goes beyond `.get`; `future` is not `Copyable`, while `shared_future` is `Copyable`.

What happens if you call `.get` twice on a `future`?

```
promise<string> p;  
future<string> f = p.get_future();  
p.set_value("hello");  
  
int a = f.get();  
int b = f.get();
```

The answer is undefined behavior. Today:

- [`libc++` seg faults..](#)
- [`libstdc++` throws an exception..](#)
- [`Boost` prints "hello" and then "" \(non-conforming\)..](#)

While I do find undefined behavior and the implementation divergence that stems from it troubling, I think there's a bigger problem. `.get` is not a read-only accessor; it is a destructive, mutating operation. This is unintuitive! The user didn't ask to move out of `f`, e.g. they didn't write:

```

promise<string> p;
future<string> f = p.get_future();
p.set_value("hello");

int a = move(f).get();
int b = f.get();

```

By using `shared_future` instead of `future` in the above examples, we get the behavior we desire.

We have the same limitation with `.then`. A `future` may only have a single continuation attached to it. E.g. the following is undefined behavior:

```

promise<void> p;
future<void> f = p.get_future();
p.set_value();

future<int> a = f.then([](future<void>) { return 1; });
future<int> b = f.then([](future<void>) { return 2; });

```

Again, the above example works fine if you use `shared_future`.

Alternatively, we could have a `future` like this:

```

template <typename T>
struct future
{
    T const& get() const&;
    T& get() &;
    T get() &&;

    template <typename F>
        auto then(F&& func) &;
    template <typename F>
        auto then(F&& func) &&;
};

```

Such a `future` would still support non-`Copyable` types, as long as you don't try to copy the `future`. `.then` continuations that take their `future` by value are still optimal, since the non-rvalue-reference overloads return by reference:

```

future<string> f = /* ... */;

move(f).then(
    [](future<string> v) // v is move constructed from f.
    {
        return v.get() + "\n"; // T& get() & called; no copy.
    }
);

```

```

future<string> g = /* ... */;

g.then(
    [](future<string> v) // v is copy constructed from f.
    {
        return v.get() + "\n"; // T& get() & called; no copy.
    }
);
g.then(/* ... */);

```

Alternatively, a more explicit interface for operations that move out of the shared state could be adopted:

```

template <typename T>
struct future
{
    T const& get() const&;
    T& get();

    T move();

    template <typename F>
        auto then(F&& func);

    template <typename F>
        auto move_then(F&& func);
};

```

By combining `future` and `shared_future` into a single type, we can move away from single consumer by default (which is often not what users want) while still providing the capability to move out of the shared state upon request.

Admittedly, both of my suggested approaches have downsides as well, when compared with the current model. With both approaches, you can move out of the shared state while other consumers (e.g. other `future`s pointing to the same shared state) still exist. The unique ownership model of `future` makes this much harder to do today.

§ 2.2. Concurrent Producer/Consumer(s) vs Non-Concurrent Producer/Consumer(s)

Both `future` and `shared_future` are designed to support scenarios where the producer of an asynchronous value is a different thread of execution than the consumer of an asynchronous value and both threads may concurrently access the shared state.

While this is the common case for `future/shared_future`s when they are used in asynchronous tasking systems (e.g. `async`-like interfaces), it pessimizes users who want to produce and consume asynchronous values within a single thread (e.g. lazy evaluation). This is a common pattern in applications interacting with a network or with user input. `future/shared_future` are typically implemented using heavyweight synchronization primitives (condition variable + mutex), and those who do not need the synchronization are always forced to pay these costs.

§ 2.3. Coupling with Scheduling and Genericity

§ 2.3.1. `future/promise` are Too Tightly Coupled to Scheduling

One issue that has come up during the evolution of executors is the need for a future concept, since some executors will need their own future type.

Why?

Notionally, a `future<T>` is a `variant` of `T` and `exception_ptr`, and a state. What prevents different executors from using the same fundamental `future` type.

The answer is that our current `future` is tightly coupled with one particular scheduling mechanism today:

- `future` needs a way to **block** so that `.get` and `.wait*` can be implemented.
- `future`'s destructor needs to block if the shared state was created by `async`, the shared state is not ready and this `future` was holding the last reference to the shared state (I will refrain from discussing this particular subject in detail; I hope we can all agree that conditionally blocking in the destructor is confusing and not ideal).

Blocking is the problem here. Each executor type may have a different way to block. A typical implementation of `future` uses `condition_variable` and `mutex` for synchronization and blocking.

For a parallel tasking system like HPX, these primitives are problematic, because they block at the OS-thread level, not the task level, and thus interfere with our userspace scheduling. Thus, we need an `hpx::future`. Libraries that manage asynchronous operations on accelerators or GPUs will also need their own `future` types.

Likewise, for a networking library, we might need to check for new messages and process outstanding work items while blocking - otherwise, we might never receive and process the message that will change the state of the future we are blocking on:

```
T get()
{
    while (!ready())
    {
        // Poll my endpoint to see if I have any messages;
        // If so, enqueue the tasks described by the messages.
        check_messages();

        // If my task queue is not empty, dequeue some tasks
        // and execute them.
        run_tasks();
    }
    return shared_state->get();
}
```

Thus, we need our own `future` type.

If we could either remove blocking interfaces from `future/promise` or parameterize the blocking mechanism, I suspect we could create a `future/promise` that could be used by most executors. There is still the issue of synchronizing the shared state between consumers and the producer, but this could be done with atomics. I believe this would be fine for parallel tasking systems, but atomics would still introduce unnecessary overheads for the non-concurrent producer/consumer use case.

§ 2.3.2. `future/promise` are Not Tightly Enough Coupled to Scheduling

On the other hand, `future/promise` are not enough coupled to scheduling. `future` often serves a dual role as a handle to both a value and an implicit handle to the task that will produce that value.

However, there are some operations that we may wish to perform on a task, such as cancellation, that are not supported by the current `future`.

§ 2.3.3. Where Does a `.then` Continuation Run?

In our current pre-executor world, it is unspecified where a `.then` continuation will be run. There are a number of possible answers:

- If the shared state is ready when the continuation is set, the consumer thread executes the continuation.
- If the shared state is not ready when the continuation is set, the producer thread executes the continuation.
- Alternatively, a new thread could be created which executes the continuation.

Executors do not entirely alleviate this problem; the question simply becomes "Where does a `.then` continuation get passed to the executor?"

Consider, for example, an executor that always enqueues a work item into a task queue associated with the current OS-thread. If the continuation is added to the executor on the consumer thread, the consumer thread will execute it. Otherwise, the producer thread will execute the continuation. Should people simply avoid writing such executors?

There is also the question of what should be done in the case that an executor parameter is omitted. I favor the third option, of spawning a new thread that executes the continuation.

§ 2.4. Composability

§ 2.4.1. Passing `futures` to `.then` Continuations is Unwieldy

The signature for `.then` continuations in the Concurrency TS v1 is:

```
ReturnType( future<T> )
```

The `future` gets passed to the continuation instead of the value so that continuation can handle `futures` that contain exceptions. The `future` passed to the continuation is always ready; `.get` can be used to retrieve the value, and will not block. Unfortunately, this can make `.then` quite unwieldy to work with, especially when you want to use existing functions that cannot be modified as continuations:

```
future<double> f;
future<double> f.then(abs); // ERROR: No std::abs(future<double>) overload.
future<double> f.then([](future<double> v) { return abs(v.get()); }); // OK
```

`future`s would be far more composable if `.then` passed the value to the continuation instead of the future itself. There are a few ways that exceptions could be handled if `.then` behaved this way:

- If the `future` that the continuation is being attached to contains an exception, the exception could be propagated to the future returned by `.then` and the continuation could never run.
- `.then` could invoke the continuation with the `exception_ptr` contained in the `future` in the event of an error if the continuation is invocable with an `exception_ptr`. Otherwise, the exception could be propagated as described above.
- `.then` could take a second, optional parameter - a `Callable` that will be invoked with the `future`'s exception in the case of an error. Otherwise, the exception could be propagated as described above.

§ 2.4.2. `when_all` and `when_any` Return Types are Unwieldy

`when_all` has the following signature (Concurrency TS v1, 2.7 [`futures.when_all`] p2):

```
template <typename InputIterator>
future<vector<typename iterator_traits<InputIterator>::value_type>>
when_all(InputIterator first, InputIterator last);

template <typename... Futures>
future<tuple<decay_t<Futures>...>>
when_all(Futures&&... futures);
```

And `when_any` has the following signature (Concurrency TS v1, 2.9 [`futures.when_any`] p2):

```

template <typename Sequence>
struct when_any_result
{
    std::size_t index;
    Sequence futures;
};

template <typename InputIterator>
future<when_any_result<vector<typename iterator_traits<InputIterator>::value_type>>>
when_any(InputIterator first, InputIterator last);

template <typename... Futures>
future<when_any_result<tuple<decay_t<Futures>...>>>
when_any(Futures&&... futures);

```

The TL;DR version:

- `when_all` either returns a `future<vector<future<T>>>` or a `future<tuple<future<Ts>...>>`.
- Likewise for `when_any`, with the added complication of the future value type being wrapped in `when_any_result`, which really wants to be a `variant` instead.

Again, the reason for the complexity here is error reporting. If `when_all`'s return type was simplified from `future<vector<future<T>>>` to `future<vector<T>>`, what would we do if some of the `futures` being combined threw exceptions? One possible answer would be for `.get` on the result of `when_all` to throw something like an `exception_list`, where each element of the list would be a `tuple<size_t, exception_ptr>`. An error that occurs during the combination of the `futures` (e.g. in `when_all` itself) could be distinguished by using a distinct exception type.

One benefit of this simplification is that it would enable this pattern:

```

bool f(string, double, int);

future<string> a = /* ... */;
future<double> b = /* ... */;
future<int>    c = /* ... */;

future<bool> d = when_all(a, b, c).then(
    [](future<tuple<int, double, string>> v)
    {
        apply(f, v); // f(a.get(), b.get(), c.get());
    }
);

```

If `.then` passed a value to the continuation instead of a `future`, this would become:

```

future<bool> d = when_all(a, b, c).then(
    [](tuple<int, double, string> v)
    {
        apply(f, v); // f(a.get(), b.get(), c.get());
    }
);

```

`.then` could be extended even further, to handle `future<tuple<Ts...>>` specially:

```

future<bool> d = when_all(a, b, c).then(f); // f(a.get(), b.get(), c.get())

```

Alternatively, the above example could be enabled by making `future` variadic and making `when_all`'s heterogeneous signature:

```

template <typename... Futures>
future<Futures::value_type...>
when_all(Futures&&... futures);

```

`when_any`, clearly, can be updated to use `variant`, which is a natural fit for its interface.

`.then` could be extended to have `visit` like semantics for `when_any future`s (e.g. `future<variant<Ts...>>`) in the same way that `.then` could be extended to have `apply` like

semantics for `when_all`.

§ 2.4.3. Immediate Values

In C++11, there was no convenience function for creating a future that is ready and contains a particular value (e.g. **immediate values**). You'd have to write:

```
promise<string> p;  
future<string> a = p.get_future();  
p.set_value("hello");
```

The Concurrency TS v1 adds such a function, `make_ready_future` (Concurrency TS v1, 2.10 [futures.make_ready_future]):

```
future<string> a = make_ready_future("hello");
```

However, it is still unnecessarily verbose to work with immediate values. Consider:

```
bool f(string, double, int);  
  
future<string> a = /* ... */;  
future<int> c = /* ... */;  
  
future<bool> d = when_all(a, make_ready_future(3.14), c).then(/* Call f. */
```

Why not allow both `future` and non-`future` arguments to `when_all`? Then we could write:

```
future<bool> d = when_all(a, 3.14, c).then(/* Call f. */);
```

In combination with the direction described in the previous section, we'd be able to write:

```
future<bool> d = when_all(a, 3.14, c).then(f); // f(a.get(), 3.14, c.get())
```

Additionally, with C++17 class template deduction, instead of `make_ready_future`, we could just have a ready `future` constructor:

```
auto f = future(3.14);
```

§ 3. HPX/Berkeley Inspired Future

This `future` design is based on some ideas from HPX and a few projects that use `future`s at my work place. The goal is to create a future without blocking interfaces that can be used by different executors in lieu of a single unified future concept. The design has evolved a lot in the past few weeks, so please forgive the rough edges.

Note that `when_any` is omitted due to time constraints; I hope to have it hammered out for the Toronto meeting. This design does not surface the notion of a task, unlike Sean Parent's design from `stlab`; I am hoping to address this (and cancellation) in the future.

First, some example usage:

```
// Ready futures.
auto g = future();           // value_type = void.
auto h = future("hello");    // value_type = string.
auto i = future("hello", 3.14, 17); // value_type = tuple<string, double, int>.

// Continuations.
auto j = g.then(F);          // async(F) AKA f().
auto k = h.then(F);          // async(F, h.get()) AKA f(h.get()).
auto l = move(h).then(F);    // async(F, h.move()) AKA f(h.move()).
auto m = h.move_then(F);     // async(F, h.move()) AKA f(h.move()).
auto n = future(3).then(F);  // async(F, future(3).move()) AKA F(future(3).move()).

// Apply-style Continuations.
auto o = i.then(F); // F("hello", 3.14, 17).

// Continuation chaining.
auto p = async(D).then(E).then(F); // F(E(D())); all values moved.
auto q = h.then(E).then(F);        // F(E(g)); all values except g moved.
```

```
future<string> g = /* ... */;
future<double> h = /* ... */;
future<int> i = /* ... */;

// Combining.
auto j = when_all(g, h, i); // value_type = tuple<string, double, int>.
auto k = when_all(g, 3.14, i); // value_type = tuple<string, double, int>.

// Apply-style Continuations.
auto l = when_all(g, h, i).then(F); // F(g.get(), h.get(), i.get())
auto m = when_all(g, 3.14, i).then(F); // F(g.get(), 3.14, i.get())
auto n = move(when_all(g, h, i)).then(F); // F(g.move(), h.move(), i.get())
auto o = when_all(g, h, i).move_then(F); // F(g.move(), h.move(), i.get())
auto p = when_all(move(g), h, i).move_then(F); // F(g.move(), h.get(), i.get())
```

Here's a (rough) synopsis of the interface:

```

namespace std2
{

template <typename... Ts>
struct future
{
    using value_type =
        // void          when sizeof...(Ts) == 0 (e.g. future<>).
        // Ts...         when sizeof...(Ts) == 1 (e.g. future<T>).
        // tuple<Ts...> when sizeof...(Ts) > 1 (e.g. future<T, ...>).

    using reference      = value_type&;
    using const_reference = value_type const&;

    // DefaultConstructible.
    constexpr future() noexcept = default;

    // MoveAssignable.
    //
    // Requires: Each Ts shall be MoveAssignable.
    future(future&& other) noexcept = default;
    future& operator=(future&& other) noexcept = default;

    // CopyAssignable.
    //
    // Requires: Each Ts shall be CopyAssignable.
    future(future&& other) noexcept = default;
    future& operator=(future&& other) noexcept = default;

    //////////////////////////////////////
    // Composition.

    // when_all and make_ready_future constructor.
    //
    // Effects: Constructs a future that will be ready when all of us are
    // ready; if us is not a future, it is ready.
    //
    // Requires:
    // * sizeof...(Us) == sizeof...(Ts)
    // * The result of get<0>(tuple_cat(detail::move_or_get(us)...)) shall
    //   assignable to value_type when sizeof...(Ts) == 1.
    // * The result of tuple_cat(detail::move_or_get(us)...) shall be
    //   assignable to value_type when sizeof...(Ts) > 1.

```

```

template <typename... Us>
future(Us&&... us);

/////////////////////////////////////////////////////////////////
// Continuation Attachment.

// Effects: Equivalent to:
// * return async(forward<Continuation>(cont));
//   when sizeof...(Ts) == 0.
// * return async(forward<Continuation>(cont), get());
//   when sizeof...(Ts) == 1.
// * return apply(async, tuple_cat(tuple(forward<Continuation>(cont)),
//   when sizeof...(Ts) > 1.
template <typename Continuation>
auto move_then(Continuation&& cont);
template <typename Continuation>
auto then(Continuation&& cont) &&;

// Effects: Equivalent to:
// * move(); return async(forward<Executor>(exec), forward<Continuation>
//   when sizeof...(Ts) == 0.
// * return async(forward<Executor>(exec), forward<Continuation>(cont),
//   when sizeof...(Ts) == 1.
// * return apply(async,
//   tuple_cat(tuple(forward<Executor>(exec), forward<Continuation>
//   move());
//   when sizeof...(Ts) > 1.
template <typename Executor, typename Continuation>
auto move_then(Executor&& exec, Continuation&& cont);
template <typename Executor, typename Continuation>
auto then(Executor&& exec, Continuation&& cont) &&;

// Effects: Equivalent to:
// * return async(forward<Continuation>(cont));
//   when sizeof...(Ts) == 0.
// * return async(forward<Continuation>(cont), get());
//   when sizeof...(Ts) == 1.
// * return apply(async, tuple_cat(tuple(forward<Continuation>(cont)),
//   when sizeof...(Ts) > 1.
template <typename Continuation>
auto then(Continuation&& cont);

// Effects: Equivalent to:

```

```

// * return async(forward<Executor>(exec), forward<Continuation>(cont))
//   when sizeof...(Ts) == 0.
// * return async(forward<Executor>(exec), forward<Continuation>(cont),
//   when sizeof...(Ts) == 1.
// * return apply(async,
//   tuple_cat(tuple(forward<Executor>(exec), forward<Cor
//   get()));
//   when sizeof...(Ts) > 1.
template <typename Executor, typename Continuation>
auto then(Executor&& exec, Continuation&& cont);

////////////////////////////////////
// Status Observers.

// Returns: true if the future is ready.
bool ready() const;

// Returns: true if the future is associated with a shared state.
bool valid() const;

////////////////////////////////////
// Consumer Access.

// Effects: Moves the value out of the shared state and invalidates the
// shared state.
//
// Precondition: ready() is true and valid() is true.
//
// Throws: The stored exception, if an exception was stored in the shar
// state.
value_type move();
value_type get() &&;

// Effects: Returns a reference to the value of the shared state.
//
// Precondition: ready() is true and valid() is true.
//
// Throws: The stored exception, if an exception was stored in the shar
// state.
reference get();
const_reference get() const;
};

```

```

template <typename T>
struct future_result
{
    using type = T;
};

template <typename... Ts>
struct future_result<future<Ts...>>
{
    using type = typename future<Ts...>::value_type;
};

template <typename T>
using future_result_t = typename future_result<T>::type;

// Deduction guide for implicit unwrapping.
template <typename... Us>
future(Us&&... us) -> future<future_result_t<Us>...>;

namespace detail
{

// Effects: Equivalent to return t.get();.
template <typename T>
auto move_or_get(future<T>& t);

// Effects: Equivalent to return t.get();.
template <typename T>
auto move_or_get(future<T> const& t);

// Effects: Equivalent to return t.move();.
template <typename T>
auto move_or_get(future<T>&& t);

// Effects: Equivalent to return forward<T>(t);
template <typename T>
auto move_or_get(T&& t);

}

template <typename... Ts>
struct promise
{

```

```

// DefaultConstructible.
constexpr promise() noexcept;

// MoveAssignable.
promise(promise&& other) noexcept = default;
promise& operator=(promise&& other) noexcept = default;

// Returns: The future attached to this promise's shared state.
//
// Precondition: get_future has not been called previously on this
// promise..
future<Ts...> get_future();

// Effects: Sets the promise's value and changes its state to ready.
//
// Requires:
// * sizeof...(Ts) == sizeof...(Us)
// * Each Ts shall be constructible from the corresponding element of Us
//
// Precondition: The promise is not ready.
template <typename... Us>
void set_value(Us&&... us);

// Effects: Sets the promise's to ready and sets its value to error.
//
// Precondition: The promise is not ready.
void set_exception(exception_ptr error);
};

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

// Effects: Adds a work item to exec (or the default global spawner) which
// calls invoke(f, args...) and returns a future that is ready when the
// work item is completed (e.g. when cont has been executed). The result of
// invoke(f, args...) is the value of the future.
//
// Requires: is_invocable_v<F, Args...> is true.
template <typename F, typename... Args>
future<decltype(declval<F>()(declval<Args>()))>
async(F&& f, Args&&...);

// Note: This function shall not participate in overload resolution unless
// is_executor_v<Executor> is true.

```

```

template <typename Executor, typename F, typename... Args>
future<decltype(declval<F>()(declval<Args>()))>
async(Executor&& exec, F&& f, Args&&...);

/////////////////////////////////////////////////////////////////

// Effects: Equivalent to: return future(forward<Us>(us)...);
template <typename... Us>
future<future_result_t<Us>...>
when_all(Us&&... us);

// Effects: Constructs a future that is ready when all the elements in
// [first, last) are ready.
template <typename InputIterator>
future<vector<future_result_t<iterator_traits<InputIterator>::value_type>...>
when_all(InputIterator first, InputIterator last);

}

```

§ 4. Acknowledgements

A big thanks to David Sankel and Sean Parent, who have provided a lot of useful insight over the past few weeks.

§ References

§ Informative References

[P0443r1]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysisen, Carter Edwards, Gordon Brown. [A Unified Executors Proposal for C++](https://wg21.link/p0443r1). 6 January 2017. URL: <https://wg21.link/p0443r1>